On the other hand, unit specs are typically too low-level to be of much use during large-scale refactoring. In fact, you may even need to throw some unit specs away during such a refactoring. Since they're meant to be cheap to write and run, that shouldn't feel like a big sacrifice.

Integration Specs

Integration specs sit somewhere between these two extremes. Code that interacts with an external service—such as a database or third-party REST API—should have an integration spec.

There's a fine line to draw here. Any nontrivial software project will depend on other libraries. A strict interpretation of these definitions would require you to separate your unit and integration specs as follows:

- Your unit specs would have to isolate your code from *any* third-party dependency.
- Your (comparatively slow) integration specs would be allowed to hit third-party code indirectly.

We recommend applying common sense here. If your Ruby class depends on small, stable, fast libraries that don't hit the network and don't have side effects, it's probably okay to call them as is from your unit specs.

Integration specs are often an order of magnitude slower than unit specs. Consequently, you most likely won't run them constantly, the way you might with unit specs. We do, however, recommend running integration specs when you modify the code they cover, or at the very least before you commit your changes.

Moreover, integration specs require more care to avoid interference. When your code writes a database record, its spec will need to delete the record (or roll back the database transaction) so as not to affect the subsequent examples. These sorts of dependency issues make integration specs more difficult to run in parallel (something that can save you a lot of time) than unit specs.

We recommend putting as little branching logic as possible in the sections of your code that deal with dependencies. The simpler these parts of your system are, the fewer expensive integration specs you'll need to check them thoroughly.

Guidelines

With these definitions in mind, we've adopted a set of guidelines for the projects in this book. They're an adaptation of principles that have served us well in real-world situations. That said, they may not be applicable for every